Homework – 5

1)Use linked list for polynomial representation and perform addition and subtraction operations on the performance.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int coeff, exp;
   struct Node *next;
};
struct Node *create(int c, int e) {
   struct Node *n = malloc(sizeof(struct Node));
   n->coeff = c;
   n->exp = e;
   n->next = NULL;
   return n;
}
void insert(struct Node **head, int c, int e) {
   struct Node *n = create(c, e);
   if (*head == NULL) {
      *head = n;
      return;
```

```
    }
    struct Node *t = *head;
    while (t->next != NULL)
        t = t->next;
    t->next = n;
}
void display(struct Node *p) {
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
        p = p->next;
        if (p != NULL) printf(" + ");
    }
    printf("\n");
}
struct Node *calculate(struct Node *a, struct Node *b, int sign) {
    struct Node *result = NULL;
    while (a != NULL && b != NULL) {
        if (a->exp == b->exp) {
            int c = a->coeff + sign * b->coeff;
            if (c != 0) insert(&result, c, a->exp);
            a = a->next;
```

```
            b = b->next;
        } else if (a->exp > b->exp) {
            insert(&result, a->coeff, a->exp);
            a = a->next;
        } else {
            insert(&result, sign * b->coeff, b->exp);
            b = b->next;
        }
    }
    while (a != NULL) {
        insert(&result, a->coeff, a->exp);
        a = a->next;
    }
    while (b != NULL) {
        insert(&result, sign * b->coeff, b->exp);
        b = b->next;
    }
    return result;
}
struct Node *readPoly() {
    struct Node *head = NULL;
```

```
    int n, c, e;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d%d", &c, &e);
        insert(&head, c, e);
    }
    return head;
}
int main() {
    struct Node *a, *b, *result;
    printf("First polynomial\n");
    a = readPoly();
    printf("Second polynomial\n");
    b = readPoly();
    printf("Polynomial 1: ");
    display(a);
    printf("Polynomial 2: ");
    display(b);
    result = calculate(a, b, 1);
```

```
    printf("Addition: ");
    display(result);
    result = calculate(a, b, -1);
    printf("Subtraction: ");
    display(result);
    return 0;
}
```

OUTPUT:

First polynomial

Enter number of terms: 3

Enter coefficient and exponent: 5 2

Enter coefficient and exponent: 3 1

Enter coefficient and exponent: 2 0

Second polynomial

Enter number of terms: 3

Enter coefficient and exponent: 2 2

Enter coefficient and exponent: 1 1

Enter coefficient and exponent: 4 0

Addition: 7x^2 + 4x^1 + 6x^0

Subtraction: 3x^2 + 2x^1 + -2x^0

2.Find previous node of a circular linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
   int data;
   struct Node *next;
};
int main() {
   struct Node *head = NULL, *temp, *n;
   int size, i, key;
   printf("Enter number of nodes: ");
   scanf("%d", &size);
   if (size <= 0) {
      printf("List is empty");
      return 0;
   }
   for (i = 0; i < size; i++) {
      n = malloc(sizeof(struct Node));
      printf("Enter data: ");
      scanf("%d", &n->data);
      n->next = NULL;
```

```
    if (head == NULL) {
        head = n;
        temp = n;
    } else {
        temp->next = n;
        temp = n;
    }
}
temp->next = head;
printf("Enter the node value: ");
scanf("%d", &key);
temp = head;
do {
    if (temp->next->data == key) {
        printf("Previous node = %d", temp->data);
        return 0;
    }
    temp = temp->next;
} while (temp != head);
printf("Node not found");
return 0;
```

```
}
```

OUTPUT:

Enter number of nodes: 4

Enter data: 10

Enter data: 20

Enter data: 30

Enter data: 40

Enter the node value: 30

Previous node = 20

3)Insert at a given position of a circular linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
int main() {
    struct Node *head = NULL, *temp, *n;
    int size, i, value, pos;
    printf("Enter number of nodes: ");
    scanf("%d", &size);
```

```
for (i = 0; i < size; i++) {
   n = malloc(sizeof(struct Node));
   printf("Enter data: ");
   scanf("%d", &n->data);
   n->next = NULL;
   if (head == NULL) {
      head = n;
      temp = n;
   } else {
      temp->next = n;
      temp = n;
   }
}
if (head != NULL)
   temp->next = head;
printf("Enter value to insert: ");
scanf("%d", &value);
printf("Enter position: ");
scanf("%d", &pos);
n = malloc(sizeof(struct Node));
n->data = value;
```

```
if (pos < 1 || pos > size + 1) {
    printf("Invalid position");
    free(n);
    return 0;
}
if (head == NULL) {
    n->next = n;
    head = n;
} else if (pos == 1) {
    temp = head;
    while (temp->next != head)
        temp = temp->next;
    n->next = head;
    temp->next = n;
    head = n;
} else {
    temp = head;
    for (i = 1; i < pos - 1; i++)
        temp = temp->next;
    n->next = temp->next;
    temp->next = n;
```

```
    }
    printf("Circular linked list: ");
    temp = head;
    if (head != NULL) {
        do {
            printf("%d ", temp->data);
            temp = temp->next;
        } while (temp != head);
    }
    return 0;
}
```

OUTPUT:

Enter number of nodes: 4

Enter data: 10

Enter data: 20

Enter data: 30

Enter data: 40

Enter value to insert: 15

Enter position: 2

Circular linked list: 10 15 20 30 40

4)Find largest second element in a Queue.

```
#include <stdio.h>
int main() {
    int q[100], n, i;
    int largest, second;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n < 2 || n > 100) {
        printf("Invalid number of elements");
        return 0;
    }
    printf("Enter queue elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &q[i]);
    }
    largest = q[0];
    second = q[0];
    for (i = 1; i < n; i++) {
        if (q[i] > largest) {
            second = largest;
            largest = q[i];
        } else if (q[i] < largest &&
```

```
            (second == largest || q[i] > second)) {
        second = q[i];
      }
    }
    if (largest == second) {
      printf("No distinct second-largest element");
    } else {
      printf("Second largest = %d", second);
    }
    return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter queue elements: 10 40 20 50 30

Second largest = 40

5)Insert 3rd element and check whether the 1st and 3rd element are same in linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
```

```
    struct Node *next;
};
int main() {
    struct Node *head = NULL, *temp, *n;
    int size, i, value;
    printf("Enter number of nodes: ");
    scanf("%d", &size);
    for (i = 0; i < size; i++) {
        n = malloc(sizeof(struct Node));
        printf("Enter data: ");
        scanf("%d", &n->data);
        n->next = NULL;
        if (head == NULL) {
            head = n;
            temp = n;
        } else {
            temp->next = n;
            temp = n;
        }
    }
    if (size < 2) {
```

```
    printf("At least 2 nodes are required");
    return 0;
}
printf("Enter value to insert at 3rd position: ");
scanf("%d", &value);
n = malloc(sizeof(struct Node));
n->data = value;
temp = head;
for (i = 1; i < 2; i++)
    temp = temp->next;
n->next = temp->next;
temp->next = n;
printf("Linked list: ");
temp = head;
while (temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}
if (head->data == head->next->next->data)
    printf("\n1st and 3rd elements are same");
else
```

```
        printf("\n1st and 3rd elements are not same");
    return 0;
}
```

OUTPUT:

Enter number of nodes: 3

Enter data: 10

Enter data: 20

Enter data: 30

Enter value to insert at 3rd position: 10

Linked list: 10 20 10 30

1st and 3rd elements are same

6)Implement two stacks in one array . Write routines to traverse the array and display.

```
#include <stdio.h>
#define MAX 10
int a[MAX];
int top1 = -1;
int top2 = MAX;
void push1(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
```

```
    } else {
        a[++top1] = value;
    }
}
void push2(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
    } else {
        a[--top2] = value;
    }
}
void display1() {
    int i;
    printf("Stack 1: ");
    for (i = top1; i >= 0; i--)
        printf("%d ", a[i]);
    printf("\n");
}
void display2() {
    int i;
    printf("Stack 2: ");
```

```
    for (i = top2; i < MAX; i++)
        printf("%d ", a[i]);
    printf("\n");
}
int main() {
    int n1, n2, i, value;
    printf("Enter number of elements in Stack 1: ");
    scanf("%d", &n1);
    printf("Enter Stack 1 elements: ");
    for (i = 0; i < n1; i++) {
        scanf("%d", &value);
        push1(value);
    }
    printf("Enter number of elements in Stack 2: ");
    scanf("%d", &n2);
    printf("Enter Stack 2 elements: ");
    for (i = 0; i < n2; i++) {
        scanf("%d", &value);
        push2(value);
    }
    display1();
```

```
    display2();
    return 0;
}
```

OUTPUT:

Enter number of elements in Stack 1: 3

Enter Stack 1 elements: 10 20 30

Enter number of elements in Stack 2: 2

Enter Stack 2 elements: 40 50

Stack 1: 30 20 10

Stack 2: 50 40

7)Given a linked list containing n head nodes where every node in the Linked List contains two pointers – Next points to next node in the bottom pointer to a sub linked list where the current node is the head. Each of the sub linked lists nodes and the head nodes are sorted in ascending order based on their data. We need to flatten the Linked List so that all the nodes appear in a single Linked List while maintaining sorted order.

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
    struct Node *bottom;
```

```
};
struct Node *create(int value) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = value;
    n->next = NULL;
    n->bottom = NULL;
    return n;
}
struct Node *merge(struct Node *a, struct Node *b) {
    if (a == NULL)
        return b;
    if (b == NULL)
        return a;
    struct Node *result;
    if (a->data <= b->data) {
        result = a;
        result->bottom = merge(a->bottom, b);
    } else {
        result = b;
        result->bottom = merge(a, b->bottom);
    }
```

```
    result->next = NULL;
    return result;
}
struct Node *flatten(struct Node *head) {
    if (head == NULL || head->next == NULL)
        return head;
    head->next = flatten(head->next);
    head = merge(head, head->next);
    return head;
}
int main() {
    struct Node *head = NULL, *last = NULL;
    struct Node *temp, *n;
    int count, size, i, j, value;
    printf("Enter number of head nodes: ");
    scanf("%d", &count);
    for (i = 0; i < count; i++) {
        printf("Enter number of nodes in sublist %d: ", i + 1);
        scanf("%d", &size);
        temp = NULL;
        struct Node *bottomLast = NULL;
```

```
    printf("Enter sorted elements: ");
    for (j = 0; j < size; j++) {
        scanf("%d", &value);
        n = create(value);
        if (temp == NULL) {
            temp = n;
            bottomLast = n;
        } else {
            bottomLast->bottom = n;
            bottomLast = n;
        }
    }
    if (head == NULL) {
        head = temp;
        last = temp;
    } else {
        last->next = temp;
        last = temp;
    }
}
head = flatten(head);
```

```
    printf("Flattened sorted linked list: ");
    temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->bottom;
    }
    return 0;
}
```

OUTPUT:

Enter number of head nodes: 3

Enter number of nodes in sublist 1: 3

Enter sorted elements: 5 7 8

Enter number of nodes in sublist 2: 2

Enter sorted elements: 10 20

Enter number of nodes in sublist 3: 2

Enter sorted elements: 19 22

Flattened sorted linked list: 5 7 8 10 19 20 22

8)Given a sorted linked list of different sizes. We need to merge them into a single list maintaining their sorted order.

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *next;
};
struct Node *create(int value) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = value;
    n->next = NULL;
    return n;
}
struct Node *merge(struct Node *a, struct Node *b) {
    if (a == NULL)
        return b;
    if (b == NULL)
        return a;
    struct Node *result;
    if (a->data <= b->data) {
        result = a;
        result->next = merge(a->next, b);
    } else {
        result = b;
```

```
        result->next = merge(a, b->next);
    }
    return result;
}
int main() {
    struct Node *lists[100];
    struct Node *temp, *n, *result = NULL;
    int k, size, i, j, value;
    printf("Enter number of linked lists: ");
    scanf("%d", &k);
    if (k < 1 || k > 100) {
        printf("Invalid number of lists");
        return 0;
    }
    for (i = 0; i < k; i++) {
        lists[i] = NULL;
        temp = NULL;
        printf("Enter number of nodes in list %d: ", i + 1);
        scanf("%d", &size);
        printf("Enter sorted elements: ");
        for (j = 0; j < size; j++) {
```

```
            scanf("%d", &value);
            n = create(value);
            if (lists[i] == NULL) {
                lists[i] = n;
                temp = n;
            } else {
                temp->next = n;
                temp = n;
            }
        }
    }
    result = lists[0];
    for (i = 1; i < k; i++) {
        result = merge(result, lists[i]);
    }
    printf("Merged sorted linked list: ");
    temp = result;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
```

```
    return 0;
}
```

OUTPUT:

Enter number of linked lists: 3

Enter number of nodes in list 1: 3

Enter sorted elements: 1 4 7

Enter number of nodes in list 2: 2

Enter sorted elements: 2 5

Enter number of nodes in list 3: 3

Enter sorted elements: 3 6 8

Merged sorted linked list: 1 2 3 4 5 6 7 8

9)Swap Kth node from beginning with Kth node from end in a linked list.

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int data;
    struct node *next;
};
int main()
{
```

```
struct node *head = NULL, *temp, *newnode;
struct node *first, *second;
int n, i, k, value;
printf("Enter number of nodes: ");
scanf("%d", &n);
for (i = 0; i < n; i++)
{
   newnode = (struct node*)malloc(sizeof(struct node));
   printf("Enter data: ");
   scanf("%d", &newnode->data);
   newnode->next = NULL;
   if (head == NULL)
      head = newnode;
   else
      temp->next = newnode;
   temp = newnode;
}
printf("Enter K: ");
scanf("%d", &k);
if (k < 1 || k > n)
{
```

```
        printf("Invalid K");
        return 0;
    }
    first = head;
    second = head;
    for (i = 1; i < k; i++)
        first = first->next;
    for (i = 1; i < n - k + 1; i++)
        second = second->next;
    value = first->data;
    first->data = second->data;
    second->data = value;
    printf("List after swapping: ");
    temp = head;
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    return 0;
}
```

OUTPUT:

Enter number of nodes: 5

Enter data: 10

Enter data: 20

Enter data: 30

Enter data: 40

Enter data: 50

Enter K: 2

List after swapping: 10 40 30 20 50

10)Multiply two numbers represented by two Linked List.

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
   int data;
   struct node *next;
};
struct node* create()
{
   struct node *head = NULL, *temp, *newnode;
   int n, i;
```

```
    printf("Enter number of digits: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        newnode = (struct node*)malloc(sizeof(struct node));
        printf("Enter digit: ");
        scanf("%d", &newnode->data);
        newnode->next = NULL;
        if (head == NULL)
            head = newnode;
        else
            temp->next = newnode;
        temp = newnode;
    }
    return head;
}
long long convert(struct node *head)
{
    long long num = 0;
    while (head != NULL)
    {
```

```
        num = num * 10 + head->data;
        head = head->next;
    }
    return num;
}
int main()
{
    struct node *num1, *num2;
    long long a, b, result;
    printf("First number:\n");
    num1 = create();
    printf("Second number:\n");
    num2 = create();
    a = convert(num1);
    b = convert(num2);
    result = a * b;
    printf("Product = %lld", result);
    return 0;
}
```

OUTPUT:

First number:

Enter number of digits: 3

Enter digit: 1

Enter digit: 2

Enter digit: 3

Second number:

Enter number of digits: 2

Enter digit: 4

Enter digit: 5

Product = 5535